

S.No: 1	Exp. Name: <i>Power of 2</i>	Date: 2026-08-11
----------------	-------------------------------------	-------------------------

Aim:

Given a positive integer n , determine whether n is a power of 2.

Input Format:

- The first line contains a single positive integer n .

Output Format

- Print "True" if n is a power of 2. Otherwise, print "False".

Constraints

- $1 \leq n \leq 10^8$

Example:

Input:

16

Output:

True

Explanation:

Since $16 = 2^4$, it is a power of 2. Therefore, the output is "True".

Note:

- 1 is considered 2^0 , and should therefore be treated as a power of 2 (output "True").
- This question doesn't contain partial weightages, all test cases must be passed.

Source Code:

```
CTP109281.py

def is_power_of_two(n):
    return n>0 and (n&(n-1))==0
n=int(input())
if is_power_of_two(n):
    print("True")
else:
    print("False")
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
16
True

Test Case - 2
User Output
18
False

S.No: 2	Exp. Name: <i>Largest Prime Factor</i>	Date: 2026-08-11
----------------	---	-------------------------

Aim:

Given a positive integer N , find and print its largest prime factor.

A prime factor is a prime number that divides N exactly without leaving a remainder.

Input Format

- The first line contains a single integer N .

Output Format

- Print the largest prime factor of N .

Constraints

- $2 \leq N \leq 10^{12}$

Example

Input

13195

Output

29

Explanation

The prime factors of 13195 are 5, 7, 13, and 29. Hence, the largest prime factor is 29.

Hint: If N is a prime number, then its largest prime factor is N itself.

Source Code:

CTP109280.py

```
def largest_prime_factor(n):
    largest=1
    i=2
    while i*i<=n:
        while n%i==0:
            largest=i
            n=n//i
        i+=1
    if n>1:
        largest=n
    return largest
n=int(input())
print(largest_prime_factor(n))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
13
13

Test Case - 2
User Output
13195
29

S.No: 3	Exp. Name: <i>Factorial Pattern Analysis</i>	Date: 2026-08-11
----------------	---	-------------------------

Aim:

A mathematical research team is analyzing patterns in large factorial numbers. One aspect they are particularly interested in is determining how many trailing zeros appear at the end of the factorial of a given number $n!$. Trailing zeros provide valuable insights into the behavior of factorial growth and are widely used in combinatorics and number theory.

As a member of the research team, your task is to develop an efficient program that computes the number of trailing zeros in $n!$ for a given value of n .

The Trailing Zeros are the Zeros that appear at the end of a number (factorial in this case).

Input Format:

- A single line of input contains an integer n .

Output Format:

- An integer representing the total number of trailing zeroes in the factorial of n .

Constraints

- $1 \leq n \leq 10^6$

Sample Test Case:

Input:

5

Output:

1

Explanation: $5! = 120$, which ends with 1 trailing zero.

Source Code:

CTP109282.py

```
def counttz(n):
    t=5
    count=0
    while(n//t!=0):
        count=count+(n//t)
        t*=5
    return count
n=int(input())
print(counttz(n))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
1

Test Case - 2
User Output
20
4

Test Case - 3
User Output
100
24

S.No: 4	Exp. Name: <i>Longest Subarray With Sum K</i>	Date: 2026-08-11
----------------	---	-------------------------

Aim:

Rahul is analyzing the calorie log of a fitness app. Each day records either calories consumed (positive value) or calories burned (negative value).

He wants to analyze the data to find the longest continuous period (subarray) where the net calorie balance equals a given target K .

Input Format:

- The first line contains an integer N , the number of days.
- The second line contains N space-separated integers, representing the calorie log.
- The third line contains an integer K , the target sum.

Output Format:

- Print the length of the longest subarray whose sum equals K .
- If no such subarray exists, print "0".

Constraints:

- $1 \leq N \leq 10^5$
- $-10^4 \leq arr[i] \leq 10^4$
- $-10^9 \leq K \leq 10^9$

Sample Input:

```
8
1 2 3 -2 5 -3 2 1
5
```

Sample Output:

```
5
```

Explanation:

The longest subarray [2, 3, -2, 5, -3] has sum 5 and length 5.

Source Code:

CTP79711.py

```
def longest_sub_array(arr,k):
    mp={}
    res=0
    prefsum=0
    for i in range(len(arr)):
        prefsum+=arr[i]
        if prefsum==k:
            res=i+1
        elif(prefsum-k) in mp:
            res=max(res,i-mp[prefsum-k])
        if prefsum not in mp:
            mp[prefsum]=i
    return res
n=int(input())
arr=[int(x) for x in input().split()]
k=int(input())
print(longest_sub_array(arr,k))
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output

8

1 2 3 -2 5 -3 2 1

5

5

Test Case - 2

User Output

6

1 -1 1 -1 1 -1

0

6

Test Case - 3

User Output
5
2 4 6 8 10
7
0

Test Case - 4
User Output
7
3 1 2 1 1 1 2
4
3

Test Case - 5
User Output
5
1 2 3 4 5
9
3

Test Case - 6
User Output
8
-1 -1 1 2 3 -2 4 1
3
5

S.No: 5	Exp. Name: <i>Stack Implementation using Arrays</i>	Date: 2026-08-18
----------------	--	-------------------------

Aim:

Write a program to implement a stack using arrays and perform the following operations:

- 1 → Push: adds an element to the top of the stack.
- 2 → Pop: removes the top element from the stack.
- 3 → Peek: returns the top element of the stack without removing it.

Input Format

- The first line contains an integer n ($1 \leq n \leq 1000$), representing the number of operations.
- The next n lines each contain the operation code followed by the element, separated by a space.
- 1 → Push: the element to be pushed follows the operation code.
- 2 → Pop: no element follows the operation code.
- 3 → Peek: no element follows the operation code.

Output Format

- For every Peek operation, print the top element of the stack on a new line.
- After all operations are completed, print the remaining elements of the stack from top to bottom, separated by a single space.

Sample Test Case

Input:

```
6
1 2
1 3
1 4
2
3
1 5
```

Output:

```
3
5 3 2
```

Explanation

- Operation 1 is "1 2", so 2 is pushed. Stack: 2
- Operation 2 is "1 3", so 3 is pushed. Stack: 3 2
- Operation 3 is "1 4", so 4 is pushed. Stack: 4 3 2
- Operation 4 is "2", so 4 is popped. Stack: 3 2
- Operation 5 is "3", so Peek is performed. Top element is 3. Stack: 3 2
- Operation 6 is "1 5", so 5 is pushed. Stack: 5 3 2

One Peek operation was performed, so 3 is printed. The final stack is: 5 3 2

Note:

- If Pop or Peek is performed on an empty stack, proceed to the next operation without displaying or modifying anything.
- If no elements remain in the stack after all operations, print "Empty".
- Display all output only after all inputs have been taken.

Source Code:

CTP17094.py

```
stack=[]
outputs=[]
def push(stack,x):
    stack.append(x)
def pop(stack):
    if stack:
        stack.pop()
def top(stack,outputs):
    if stack:
        outputs.append(stack[-1])
n=int(input())
for i in range(n):
    op=[int(x) for x in input().split()]
    if op[0]==1:
        push(stack,op[1])
    elif op[0]==2:
        pop(stack)
    elif op[0]==3:
        top(stack,outputs)
for val in outputs:
    print(val)
if stack:
    print(*(stack[::-1]))
else:
    print("Empty")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
6
1 2
1 3

1 4
2
3
1 5
3
5 3 2

Test Case - 2
User Output
5
1 44
1 55
2
2
3
Empty

S.No: 6	Exp. Name: <i>Largest Rectangular Area in Histogram</i>	Date: 2026-08-18
----------------	--	-------------------------

Aim:

You are given an array *arr* representing the heights of bars in a histogram. Find the largest rectangular area possible in the histogram where the largest rectangle can be made of a number of contiguous bars. Each bar in the histogram has a width of 1 unit.

Input Format:

- The first line contains an integer *N*, the number of bars in the histogram.
- The second line contains *N* space-separated integers, representing the heights of bars in the histogram.

Output Format:

- Output a single integer representing the largest rectangular area possible in the histogram.

Constraints:

- $1 \leq N \leq 10^3$
- $0 \leq arr[i] \leq 10^6$

Note:

- Use a stack to find the largest rectangular area in the histogram.

Sample Input:

```
7
6 2 5 4 5 1 6
```

Sample Output:

```
12
```

Explanation:

Given the input array representing the heights of histogram bars [6, 2, 5, 4, 5, 1, 6], the largest rectangular area is 12. This area is achieved by considering the bars at indices 2, 3, and 4 with heights [5, 4, 5]. The height of the smallest bar in this group is 4, and there are 3 bars, so the area is $4 * 3 = 12$. This is the largest possible rectangular area in the given histogram.

Source Code:

CTP31050.py

```
n=int(input())
arr=list(map(int,input().split()))
stack=[]
max_area=0
for i in range(n+1):
    h=0 if i==n else arr[i]
    while stack and arr[stack[-1]]>h:
        height=arr[stack.pop()]
        if stack:
            width=i-stack[-1]-1
        else:
            width=i
        area=height*width
        max_area=max(max_area,area)
    stack.append(i)
print(max_area)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
8
6 9 3 7 2 8 12 1
16

Test Case - 2
User Output
10
34 8 3 92 7 9 2 87 4 1
92

S.No: 7	Exp. Name: <i>Generate All Subsets (Power Set) of a Distinct Integer Array</i>	Date: 2026-08-25
----------------	---	-------------------------

Aim:

Given an integer array *nums* containing distinct elements, generate all possible subsets including the empty subset(the power set). The result should not include any duplicate subsets.

Input Format

- The first line contains an integer n , representing the size of the array.
- The second line contains n space-separated integers, representing the elements of the array *nums*.

Output Format

Print each subset on a separate line using square brackets.

- Print the empty subset as `[]`.
- Print each subset enclosed in square brackets, with elements separated by a comma followed by a space. For example: `[1]`, `[1, 2]`, `[1, 2, 3]`.
- Print the subsets in increasing order of their size, starting with the empty subset.
- For subsets of the same size, print them in the order generated by considering the elements from left to right as they appear in the input array (input order).

Constraints

- $1 \leq n \leq 15$
- $-10 \leq \text{nums}[i] \leq 10$
- All elements in *nums* are distinct

Instructions: To run your custom test cases strictly map your input and output layout with the visible test cases.

Source Code:

CTP78920.py

```
n = int(input())
arr=list(map(int,input().split()))
def generate(start, current, size):
    if len(current)==size:
        print("[+", ".join(map(str, current))+"]")
        return
    for s in range(start, n):
        current.append(arr[s])
        generate(s + 1, current, size)
        current.pop()
print("[]")
for size in range(1, n+1):
    generate(0, [], size)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
1 2 3
[]
[1]
[2]
[3]
[1, 2]
[1, 3]
[2, 3]
[1, 2, 3]

Test Case - 2
User Output
3
-1 0 2
[]
[-1]
[0]

$[2]$
$[-1, 0]$
$[-1, 2]$
$[0, 2]$
$[-1, 0, 2]$

S.No: 8	Exp. Name: <i>Find the Kth Smallest and Largest Element in an Array</i>	Date: 2026-09-08
----------------	--	-------------------------

Aim:

You are working as a software engineer at a company that handles large datasets. Your team is developing a data analysis tool that often requires retrieving ranked values from a list of employee performance scores.

Your task is to determine the ***K*th smallest** and ***K*th largest** performance scores from the given list.

Input Format

- The first line contains an integer N , representing the number of employee performance scores.
- The second line contains N space-separated integers representing the performance scores of the employees.
- The third line contains an integer K , representing the rank to be determined.

Output Format

- The first line contains the K th smallest employee performance score.
- The second line contains the K th largest employee performance score.

Constraints

- $1 \leq N \leq 10^5$
- $-10^9 \leq \text{performance score} \leq 10^9$
- $1 \leq K \leq N$
- All performance scores are distinct.

Sample Input

```
6
7 10 4 3 20 15
3
```

Sample Output

```
7
10
```

Explanation

Sorted performance scores: 3 4 7 10 15 20

- The 3rd smallest performance score is 7.
- The 3rd largest performance score is 10.

Source Code:

CTP109369.py

```
n = int(input())
arr=list(map(int,input().split()))
k=int(input())
arr.sort()
smallest = arr[k - 1]
largest= arr[n - k]
print(smallest)
print(largest)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
6
7 10 4 3 20 15
3
7
10

Test Case - 2
User Output
5
5 2 8 1 9
2
2
8

Test Case - 3
User Output
6
-10 -5 -30 -1 -20 -15
4
-10

S.No: 9	Exp. Name: <i>Peak Element Identification in an Array</i>	Date: 2026-09-08
----------------	--	-------------------------

Aim:

Given an array A of n integers, 0-based index of the rightmost Peak Element..
A Peak Element is an element that is strictly greater than its adjacent element(s).

Rules for identifying a Peak Element:

- For a middle element ($0 < i < n - 1$), $A[i]$ is a peak if:
 - $A[i] > A[i - 1]$ and $A[i] > A[i + 1]$
- For the first element ($i = 0$), it is a peak if:
 - $A[0] > A[1]$
- For the last element ($i = n - 1$), it is a peak if:
 - $A[n - 1] > A[n - 2]$

If the array contains multiple Peak Elements, print the index of the rightmost Peak Element.

Input Format:

- The first line contains a single integer n , representing the number of elements in the array.
- The second line contains n space-separated integers, representing the elements of the array A .

Output Format:

- Print a single integer representing the 0-based index of the rightmost Peak Element.

Constraints:

- $1 \leq n \leq 10^4$
- $1 \leq A[i] \leq 10^6$
- No two adjacent elements are equal ($A[i] \neq A[i + 1]$)
- If $n = 1$, the single element is always considered a peak, and the answer is 0.

Sample Input:

```
4
1 2 3 1
```

Sample Output:

```
2
```

Explanation:

Array elements:

- Index 0 $\rightarrow$ 1
- Index 1 $\rightarrow$ 2
- Index 2 $\rightarrow$ 3
- Index 3 $\rightarrow$ 1

The array contains a single Peak Element at index 2 because 3 is greater than both its adjacent elements (2 and 1). Since it is also the rightmost Peak Element, the output is 2.

Source Code:

CTP110230.py
<pre>n=int(input()) A=list(map(int,input().split())) for i in range(n - 1, -1, -1): if i==0: if A[i]>A[i+1]: print(i) break elif i==n -1: if A[i]>A[i-1]: print(i) break else: if A[i] > A[i - 1] and A[i] > A[i+1]: print(i) break</pre>

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
25
100 200 150 250 200 300 250 350 300 400 350 450 400 500 450 550 500 600 550 650 600 700 650 750 700
23

Test Case - 2
User Output
4
1 2 3 1
2

Test Case - 3
User Output
15
10 20 35 50 65 80 95 110 90 70 60 50 40 30 20
7